Assignment 1 Weeks 1 & 2

Database Management Systems

Part I: Short Answer Questions

Q1

A Database Management System (DBMS) provides a structured, centralised layer for storing, querying, and managing data, including built-in support for concurrency, transactions, access control, and data integrity. A traditional file system, by contrast, stores data as raw files managed by the operating system; it offers no query language, no schema enforcement, and no built-in mechanisms for concurrent multi-user access or recovery from failure.

Concrete scenario: Consider a hospital that needs to record patient admissions, doctor schedules, and billing records. With a file system, each department would likely maintain separate flat files, making it nearly impossible to answer a query such as "list all patients treated by Dr Smith who have an unpaid bill" without writing custom programs to read and cross-reference multiple files, with no guarantee of consistency if two clerks update records at the same time. A DBMS solves all of these problems: it stores the related data in interlinked tables, supports SQL queries that span tables, enforces referential integrity (e.g. a billing record cannot reference a non-existent patient), and uses locking or MVCC to allow many users to work concurrently without corrupting data.

Q2

A strong entity has a primary key composed entirely of its own attributes and can therefore be uniquely identified without reference to any other entity. A weak entity lacks sufficient attributes to form a primary key on its own; it depends on a related strong entity (its identifying owner) for part of its identity.

The primary key of a weak entity is formed by combining its partial key (a discriminator, an attribute that is unique only among the weak entities belonging to the same owner) with the primary key of its identifying owner entity. The relationship linking a weak entity to its owner is called the identifying relationship. In ER diagrams, weak entities are drawn as double rectangles, and their identifying relationships are shown with double diamonds.

Q3

Phone number should be modelled as a multivalued attribute.

A simple attribute holds exactly one atomic value per entity instance, so it cannot accommodate a student with more than one number. A composite attribute is one that can be decomposed into sub-parts (e.g., an address broken into street, city, and pin-code); a phone number is a single atomic string and does not decompose meaningfully in this context. A multivalued attribute, however, explicitly allows an entity to have zero, one, or many values for that attribute, which matches the real-world situation where a student may have a personal mobile, a home landline,

and a work number. In a relational schema, this is typically implemented as a separate table (e.g., StudentPhone(roll_no, phone_number)) with a foreign key back to the student.

Q4

A super key is any set of one or more attributes whose combined values are guaranteed to be unique across all tuples in the relation. A candidate key is a minimal super key — a super key from which no attribute can be removed while still maintaining uniqueness. In other words, every candidate key is a super key, but not every super key is a candidate key (a super key may contain redundant attributes).

Relation: Student(roll_no, email, name, branch)

Candidate keys (minimal unique identifiers):

- roll_no — each student is assigned a unique roll number.
- email — each student has a unique institutional e-mail address.

Super keys that are not candidate keys (contain extra, non-minimal attributes):

- (roll_no, email) — unique, but removing either attribute still leaves a unique identifier.
- (roll_no, name) — unique (roll_no alone is already unique), but contains the redundant attribute name.
- (roll_no, email, name, branch) — the full set of attributes; unique but far from minimal.

Q5

Constraint violated:

The Foreign Key Constraint (also called Referential Integrity Constraint).

What it protects against:

It ensures that every value in the student_id column of the Enrollment table must correspond to an existing student_id in the Student table. This prevents orphaned records — enrollment rows that reference students who do not exist — which would make the data meaningless and queries that join the two tables unreliable.

What should happen to the insert operation:

The DBMS should reject (rollback) the insert operation and return an error to the calling application. No row should be added to the Enrollment table. This behaviour is the default action defined by the SQL standard for a foreign key violation on INSERT.

Q6

Additional structure required:

A new junction table (also called a bridge table or associative table) must be created specifically to represent the M: N relationship.

Primary key composition:

The primary key of the junction table is a composite key formed by combining the primary keys of both participating entities. For example, if Student and Course are in an M:N Enrols relationship, the junction table Enrolment has a composite primary key (student_id, course_id).

Why a column in one table is insufficient:

In a relational table, a single column holds exactly one value per row. If a student can enrol in many courses, storing course IDs in a single column of the Student table would require either repeating-group columns (violating 1NF) or a comma-separated list (which destroys atomicity and makes querying extremely cumbersome). Symmetrically, a course can be taken by many students, so neither side can act as the "one" end. The junction table sidesteps this by devoting one row per (student, course) pair, cleanly representing every combination without violating relational principles.

Part II: Long Answer — Online Food Delivery Platform

Part A:

Entities and Key Attributes

Customer

- customer_id (PK)
- name
- email
- phone_number (multivalued)
- delivery_address (composite: street, city, state, pin_code)

Restaurant

- restaurant_id (PK)
- name
- cuisine_type
- address
- contact_number

MenuItem

- item_id (PK)
- name
- description

- price
- category

Order

- order_id (PK)
- order_date
- total_amount
- delivery_status

Relationships and Cardinalities

Places (Customer — Order): 1:N

A customer can place many orders, but each order is placed by exactly one customer. This is a 1:N relationship with the foreign key customer_id residing in the Order table.

AssociatedWith (Order — Restaurant): N:1

Each order is associated with exactly one restaurant; a restaurant can receive many orders. This is effectively a 1:N relationship (Restaurant to Order), with restaurant_id as a foreign key in the Order table.

Contains (Order — MenuItem): M:N

An order may contain multiple menu items, and a given menu item can appear in many different orders. Because neither side is restricted to at most one, this is an M:N relationship, realised through a junction table OrderItem that also carries the quantity attribute.

Offers (Restaurant — MenuItem): 1:N

A restaurant can offer many menu items, but each menu item belongs to exactly one restaurant (in this system, a menu item is restaurant-specific, not shared globally). The foreign key restaurant_id is placed in the MenuItem table.

Part B

Multivalued attribute phone number on Customer

A customer may have more than one phone number (mobile, home, work). Because the attribute can hold multiple values simultaneously for a single entity instance, it qualifies as multivalued. In the relational schema, it would be extracted into a separate table, e.g. CustomerPhone(customer_id, phone_number).

Composite attribute delivery address on Customer

An address is naturally decomposable into sub-parts: street, city, state, and pin_code. Because it has meaningful components that are often queried or sorted individually (e.g. filter all customers in a city, validate pin codes), it qualifies as a composite attribute. Modelling it as a

composite rather than a single string allows fine-grained access to each component without string-parsing.

Part C

The M: N relationship between Order and MenuItem requires a junction table. The complete relational schema is:

```
OrderItem(order_id, item_id, quantity, unit_price)
```

Primary Key:

(order_id, item_id) The composite primary key uniquely identifies each line item within an order (one row per menu item per order).

Foreign Keys:

- order_id references Order(order_id)
- item_id references MenuItem(item_id)

Additional attributes:

- quantity — records how many units of a given menu item were ordered (required because the same item could be ordered multiple times within one order).
- unit_price — records the price at the time of ordering; menu prices may change later, so the historical price must be stored on the order line, not looked up from MenuItem.

Part III: ER Diagram Design

Q1

Please refer to the ER diagram image included below (or attached as a separate PNG exported from draw.io as per assignment instructions).

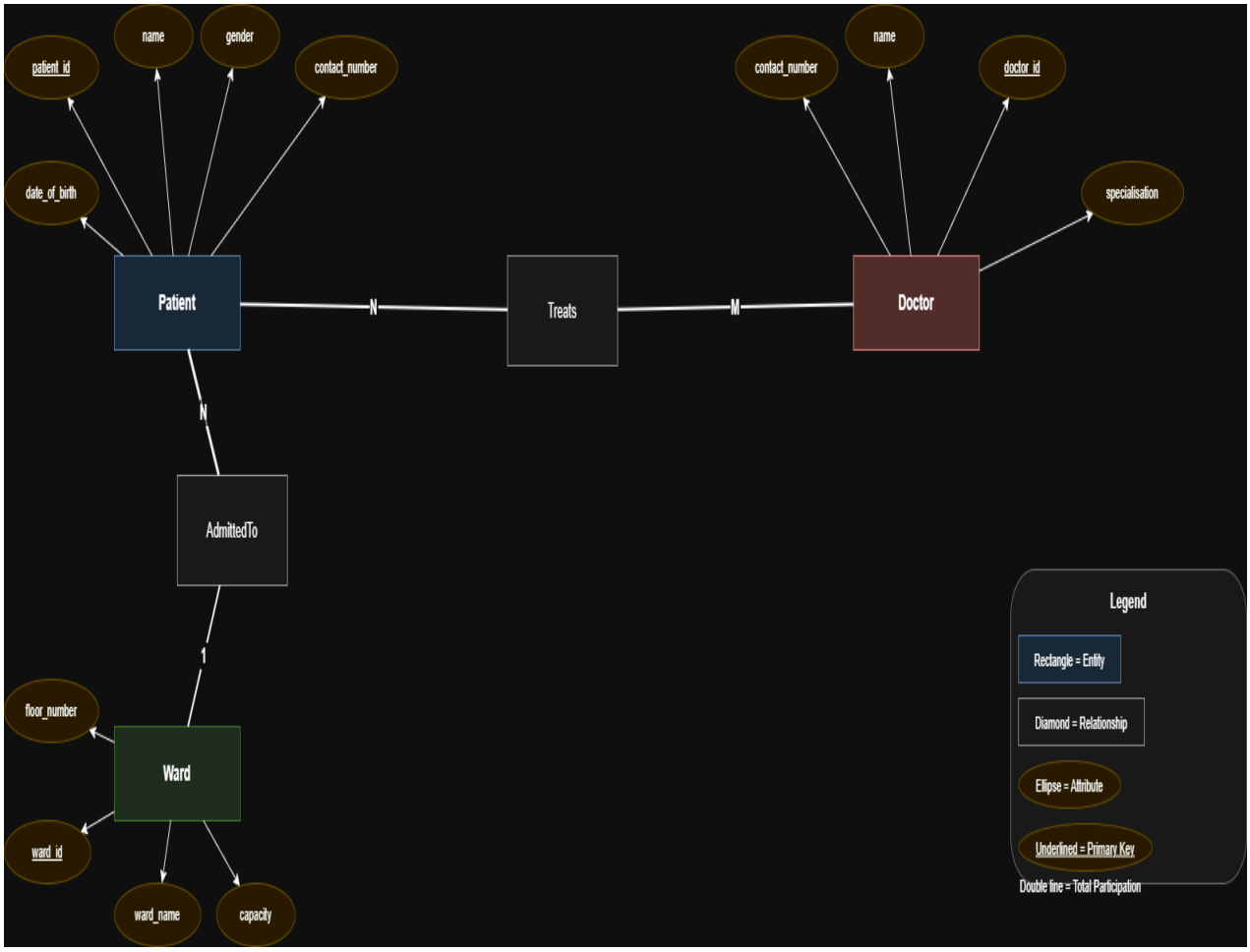
Summary of the diagram structure for reference:

Entities and attributes:

- Patient — patient_id (PK, underlined), name, date_of_birth, gender, contact_number
- Doctor — doctor_id (PK, underlined), name, specialisation, contact_number
- Ward — ward_id (PK, underlined), ward_name, capacity, floor_number

Relationships:

- AdmittedTo (Patient — Ward): Many patients can be housed in one ward (M:1/N:1). Total participation on the Patient side (every admitted patient must be assigned a ward); partial participation on the Ward side (a ward may temporarily have no patients).
- Treats (Doctor — Patient): A doctor can treat many patients, and a patient can be treated by many doctors (M: N). Partial participation on both sides.



Q2

Doctor–Patient as M: N and its associative entity

The Treats relationship between Doctor and Patient is M:N because real clinical practice involves multiple doctors attending to the same patient (e.g., a surgeon, an anaesthetist, and a

physiotherapist may all treat one patient), and each doctor simultaneously treats many patients across different wards or over time. Neither side is limited to a single counterpart, which is the defining condition for M: N. When mapped to a relational schema, this relationship produces an associative entity, a junction table named Treatment or Treats — whose composite primary key would be (doctor_id, patient_id). If the same doctor can treat the same patient on multiple occasions, a date or treatment_id attribute may be added to the primary key to distinguish individual episodes.

Participation constraints for two relationships

AdmittedTo (Patient–Ward): The Patient side requires total participation, meaning every patient in the database must be admitted to a ward. This is justified because the system is specifically for tracking admitted (in-patient) hospital stays; a patient record is created at the point of admission and therefore always has a ward assignment. The Ward side has partial participation: a ward is a physical unit of the hospital that exists in the system regardless of whether it currently has any patients. It is entirely valid to register a ward in advance of any admissions (e.g. a newly constructed wing), so a ward need not be associated with any patients at any given moment.

Treats (Doctor–Patient): Both sides have partial participation. A doctor may be registered in the system (perhaps newly hired or on leave) but not currently treating any patient, so mandatory participation on the Doctor side would be incorrect. Equally, the scenario does not restrict every patient to having a doctor assigned — an edge case might be a patient awaiting assignment — so total participation on the Patient side is not enforced either.

Attribute design decision

The address of a Doctor (or Patient) was modelled as a composite attribute decomposed into street, city, state, and pin_code rather than as a single string. This decision is justified for two reasons. First, individual sub-components are likely to be queried or grouped independently (e.g. finding all doctors in a particular city to optimise on-call scheduling). Second, composite modelling enforces data consistency: city and pin_code can be validated independently, making it far harder to accidentally store a malformed address than if the whole address were stored as a single free-text field. Had an address been unlikely to require component-level access, a single string would suffice, but the hospital context makes granular access valuable.